

Navigating Large Programming Misconception Collections Using Teaching-Relevance Criteria

Corinna List
Augsburg Technical University of
Applied Sciences
Augsburg, Germany
corinna.list@tha.de

Moritz Müller
Augsburg Technical University of
Applied Sciences
Augsburg, Germany
moritz.mueller@tha.de

Michael Kipp
Augsburg Technical University of
Applied Sciences
Augsburg, Germany
kipp@tha.de

Abstract

Despite a wealth of documented programming misconceptions, collections remain difficult to use in practice: static lists with no priority guidance leave educators without support for deciding which misconceptions deserve attention. We present a web-based misconception repository for Java/Processing designed to change this, with each entry connecting misconceptions to origins, typical errors, and concrete teaching opportunities. To support navigation, we introduce two teaching-relevance criteria: Learner Discoverability (how likely learners are to notice a misconception themselves) and Learning Barrier (how strongly it hinders progress if unaddressed). These criteria were developed iteratively using LLM-based pilot ratings to validate and refine definitions before involving human raters, reducing an initial set of five dimensions to two. Three human instructors rated all misconceptions, yielding moderate inter-rater reliability ($\alpha = 0.57$ and $\alpha = 0.60$), with a majority vote raising consistency to $\alpha = 0.9$ and $\alpha = 1.0$. Combined with observed frequency data and topic/subtopic filtering, the criteria enable sorting and compound filters (such as Silent Blockers or Common Obstacles) that translate ratings into actionable entry points for lesson preparation and tutor training, going beyond existing collections that offer no priority guidance or instructional filtering.

CCS Concepts

• **Social and professional topics** → **Computer science education**; • **Software and its engineering** → **General programming languages**.

Keywords

programming; programming languages; misconceptions; pedagogy; teacher training

ACM Reference Format:

Corinna List, Moritz Müller, and Michael Kipp. 2026. Navigating Large Programming Misconception Collections Using Teaching-Relevance Criteria. In *UK and Ireland Computing Education Research 2026 (UKICER 2026)*, September 03–04, 2026, Cambridge, United Kingdom. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3830800.3830805>



This work is licensed under a Creative Commons Attribution 4.0 International License. *UKICER 2026, Cambridge, United Kingdom*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2593-7/26/09
<https://doi.org/10.1145/3830800.3830805>

1 Introduction

Programming misconceptions, such as incorrect assumptions and incomplete mental models about how programs behave, are a well-documented challenge in introductory programming education. They can remain invisible for long periods, hinder progress, and generate substantial support demand. At the same time, they hold considerable pedagogical potential: when surfaced and addressed, they support deliberate practice [8] and catalyze reflective learning [6]. For instructors and tutors, it is therefore valuable not only to know which misconceptions commonly occur, but also how to recognize and prioritize them in everyday teaching.

Education research has documented a wide range of programming misconceptions, and several collections have emerged as shared resources [1, 3, 15]. However, these collections are typically presented as static, alphabetically ordered lists that offer no guidance on which misconceptions are most likely to affect students, which most strongly hinder progress, or which are hardest for learners to notice on their own. For an instructor preparing a lesson or training a tutor, this leaves a critical gap: knowing that a misconception exists is not the same as knowing whether it warrants instructional attention.

Consider a misconception called *Return At The End* — the misconception that a return statement must always appear at the end of a function. It is widespread among novices, strongly hinders the ability to write conditional logic in functions, and is unlikely to be noticed by learners themselves. Yet in an alphabetically sorted list, it appears indistinguishable from a minor syntactic confusion. A repository that makes instructional priorities explicit would flag this misconception immediately as frequent, hard to detect, and worth teaching against.

We therefore argue for an instructor-oriented approach to misconception collections — one that translates misconception knowledge into actionable navigation. To this end, we introduce two teaching-relevance criteria, *Learner Discoverability* and *Learning Barrier*, developed through an iterative LLM-based pilot process that surfaces definitional ambiguities before involving human raters. Combined with observed frequency data and topic/subtopic structure, these criteria are integrated into a web repository through sorting, filtering, and compound navigation mechanisms. Compound filters such as *Silent Blockers* or *Top Priorities* condense multiple criteria into instructionally meaningful categories, supporting common workflows in lesson preparation and tutor training.

We address two research questions:

RQ1: Which teaching-relevance criteria can be defined and rated reliably enough by instructors to support instructional prioritization of programming misconceptions?

RQ2: How can such criteria be integrated — through sorting, filtering, and compound filters — into a repository that supports instructors and tutors in moving from a large collection to targeted instructional decisions?

This paper makes three contributions: (1) a web-based, instructor-oriented repository for Java/Processing¹ misconceptions that links each entry to concrete teaching opportunities; (2) two teaching-relevance criteria with reliability estimates from both LLM pilot ratings and human instructor ratings; (3) a compound filter scheme that translates criteria into actionable navigation categories.

2 Related Work

Research on programming misconceptions spans three strands: empirical studies identifying misconceptions in specific topics or populations, curated collections consolidating these findings, and conceptual frameworks for reasoning about student understanding. Our work builds on all three, but focuses on a gap that cuts across them: the lack of support for *instructional prioritization*.

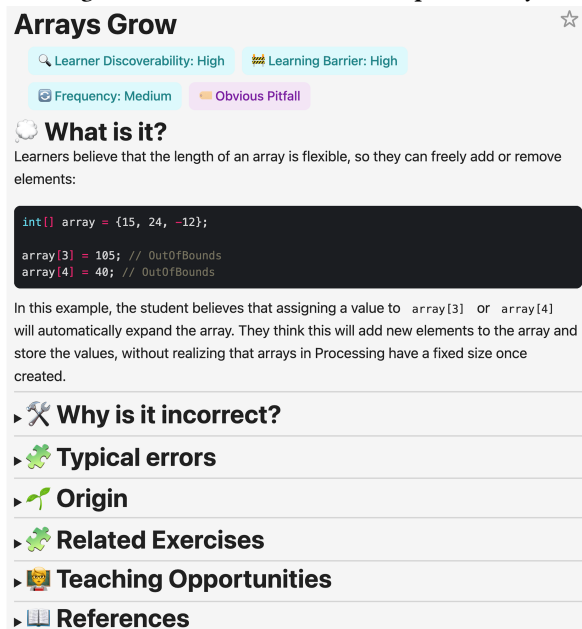
A substantial body of research has documented misconceptions in specific contexts — across programming concepts such as loops [7], objects [4], or fundamental CS concepts [12], and across languages, notably Java [1] and in broader literature reviews [15]. These studies deepen understanding of where learners struggle but are typically published as static research artifacts, not as resources designed for everyday instructional use. Our own prior work [11] falls into this category and directly motivates the present paper: identifying misconceptions is a necessary first step, but without a navigable repository, instructors struggle to act on these findings.

Several efforts consolidate misconceptions into shared resources. ProgMiscon [3] is the most comprehensive, offering structured entries with symptoms, origins, and suggested corrections, organized by language and topic tags. CSTEachingTips² links teaching tips to misconceptions via a tag-based interface. The PyTamaro misconceptions tutorial [2] offers interactive exploration of 16 Python misconceptions. While these resources vary in scope and interaction design, none offer aggregated, sortable frequency values or teaching-relevance ratings that would let instructors distinguish high-priority from peripheral misconceptions at a glance.

A third strand develops conceptual tools for reasoning about student understanding. Fincher et al.’s notional machines repository [9] documents abstractions for explaining program execution, but does not systematically link notional machines to specific misconceptions or interventions. Sorva [16] analyzes the role of mental models in introductory programming. Concept inventories such as Caceffo et al.’s Java CS1 inventory [1] use misconceptions as distractors for diagnostic assessment. These works shape how we think about student understanding but, by design, do not address the question of how instructors should prioritize misconceptions in everyday teaching.

None of the collections that we are aware of combine teaching-relevance criteria, empirical frequency data, and compound filters. Rather than primarily documenting misconceptions, our repository is designed to translate misconception knowledge into actionable navigation — helping instructors and tutors move from a large list

Figure 1: Structure of a misconception entry



of entries to a targeted starting point for lesson preparation and tutor training.

3 Components of the Misconception Repository

We designed a web-based repository³ in which each misconception is documented using a standardized template [13], developed iteratively based on the authors’ teaching experience, literature reviews, and discussions with other research groups, including contributors to the ProgMiscon [3] project.

Each entry follows a consistent structure (Figure 1): a description of the misconception (*What is it?*), an explanation of why the reasoning is incorrect (*Why is it incorrect?*), and *Typical errors* showing how the misconception manifests in student code — supporting recognition during grading or classroom interaction. *Origins* describe possible sources such as prior knowledge, instructional explanations, or misleading metaphors, identified from structured student interviews [13], informal discussions with students, tutors, and lecturers, observations of peer discussions, and the literature.

To connect misconceptions to teaching practice, each entry includes *Teaching Opportunities* — concrete interventions such as conceptual questions, short exercises, or peer instruction quizzes [14] that instructors can integrate into existing courses with minimal overhead. Peer instruction and collaborative learning have both been shown to improve learning outcomes [5, 10, 17], and several exercises are designed for small-group use. Teaching opportunities were developed iteratively within regular lectures and lab sessions of an introductory Processing course, and refined based on student responses.

¹<https://www.processing.org>

²<https://www.csteachingtips.org>

³<https://tha.de/misconceptions>

4 Defining and Refining Teaching-Relevance Criteria Using LLM-based Pilot Ratings

Documenting misconceptions is a necessary first step, but it leaves a critical question unanswered: with limited preparation and class time, instructors cannot address every misconception — so which ones matter most? To address this, we developed teaching-relevance criteria that characterize misconceptions with respect to their instructional implications and enable targeted navigation.

LLMs as Pilot Raters. Defining teaching-relevance criteria that can be applied consistently by multiple raters is itself a methodological challenge: definitions that seem clear to their authors often turn out to be ambiguous or conflated in practice. The standard approach — iterative refinement with human raters — is costly, limited by rater availability, and prone to a specific threat: raters repeatedly exposed to the rating task during definition development lose independence for the final annotation, biasing reliability estimates. We therefore propose using LLMs as pilot raters. By treating LLMs as a diverse panel of independent annotators, we can iteratively surface definitional ambiguities, identify unstable or conflated dimensions, and refine the criteria *before* involving human raters. This protects human raters’ independence and provides a fast feedback loop for definitional refinement. We emphasize that LLM ratings are *not* treated as ground truth or as a substitute for human annotation — they serve as a diagnostic tool for definitional clarity.

Initial Dimensions and Pilot Setup. Based on our previous research [13], we initially identified five candidate dimensions, each rated on a five-point scale: *Learner Correction Effort*, *Instructor Correction Effort*, *Learning Barrier*, *Learner Discoverability*, and *Instructor Discoverability*. We used five models spanning different providers and capability levels: GPT 5.1, GPT 5.0, Gemini 2.5, Gemini 3 Pro Preview, and Codestral. Each model was provided with the full written definition of each dimension, two example misconceptions with reference ratings (*Arrays Grow* and *Functions Should Be Context Specific*; excluded from the final rating set), and the complete textual description of each misconception to be rated. Each misconception was submitted as an individual query, and every model produced five independent ratings per misconception to account for response variability. Models were asked to return both a rating and a short justification.

Iterative Refinement. After each iteration, we computed inter-model reliability (Krippendorff’s α) and manually reviewed the generated justifications within the research group. The justifications served two purposes: they surfaced systematic misinterpretations pointing to definitional ambiguities, and they revealed when different dimensions were being applied interchangeably — a sign of conceptual overlap rather than just LLM confusion. Revised definitions were re-evaluated in the next iteration. Dimensions that remained unstable across multiple iterations — because they were inherently context-dependent or because justifications showed that raters (LLMs and, in subsequent manual checks by the research team, ourselves) could not reliably separate them from other dimensions — were flagged for removal.

Results of the Pilot Process. Over 15 iterations, this process refined the criteria along two levers: adjusting rating scales and removing dimensions that could not be applied consistently. Three changes had the largest effect. First, reducing the initial five-point

scale to three points raised Learner Discoverability from $\alpha = 0.34$ to $\alpha = 0.50$ — indicating that the finer scale distinctions were a source of disagreement rather than of information. Second, the correction-effort and instructor-related dimensions remained unstable regardless of scale: models underused the scales, and their justifications frequently conflated these dimensions with the learner-focused ones. We therefore removed Learner Correction Effort, Instructor Correction Effort, and Instructor Discoverability. Third, with the two remaining dimensions isolated and their definitions further refined, inter-model reliability reached $\alpha = 0.65$ (Learner Discoverability, 3-point) and $\alpha = 0.54$ (Learning Barrier, 2-point) with substantially higher consistency within individual models (each model rating the same misconception five times) than across models — that is, each model was internally stable even where models disagreed with one another (e.g., $\alpha = 0.99$ and $\alpha = 0.84$ for Gemini 3 Pro Preview). The asymmetric scale design reflects the underlying constructs: Learner Discoverability meaningfully distinguishes three levels (clearly visible / sometimes noticed / typically invisible), while Learning Barrier is most usefully treated as a binary decision about whether a misconception has instructional impact substantial enough to warrant attention. Finer distinctions on the Barrier dimension proved consistently unreliable across both LLMs and informal team rating. These values indicated that the definitions had reached sufficient clarity to proceed to human annotation. We acknowledge that LLM inter-rater reliability is not a proxy for human reliability; it is used here as a *signal* that definitions are stable enough to be worth testing with human raters.

Final Criteria.

Learner Discoverability — How likely is it that learners notice that something is wrong in their understanding or in the behavior of their program? This criterion captures how likely learners are to notice a discrepancy between their expectations and the program’s behavior, assuming that learners are given tasks in which the misconception can manifest. Learners do not identify the underlying misconception, but they may recognize that something is incorrect and seek help. Rated on a 3-point scale (High / Medium / Low).

Learning Barrier — To what extent does this misconception hinder learners in achieving current or future learning objectives? This criterion captures whether a misconception has a noticeable instructional impact if left unaddressed, distinguishing misconceptions that only affect small details from those that interfere with understanding a topic or making progress in related concepts. Rated on a 2-point scale (Low / High).

5 Human Rating of Teaching-Relevance Criteria

Rating Procedure. Three human instructors (the authors) with 2, 5, and 15 years of teaching experience in the introductory Processing course, respectively, rated all misconceptions. The procedure mirrored the LLM pilot setup (same definitions, same two calibration examples, same instruction to choose the higher value under uncertainty) so that human and model ratings are directly comparable. Raters received a written instruction with the full definitions, two example misconceptions (*Arrays Grow* and *Functions Should Be Context Specific*; excluded from the rating set) with reference values that had been agreed on in a prior calibration session, and access

to the repository. Ratings and brief justifications were recorded in a spreadsheet. To reflect the instructional principle that under-addressing a misconception is costlier than over-addressing it, raters were instructed to choose the higher value in cases of uncertainty. Where raters felt substantial uncertainty, they could mark a rating as uncertain; such ratings were excluded from the reliability analysis. Of 168 total ratings, 13 (7.7%) were marked as uncertain and removed. Inter-rater reliability was computed using Krippendorff’s α with an ordinal distance metric for Learner Discoverability and a nominal metric for the binary Learning Barrier dimension.

Reliability. The resulting coefficients indicate moderate agreement for both Learner Discoverability ($\alpha = 0.57$) and Learning Barrier ($\alpha = 0.60$). We interpret these values not as a definitional failure but as reflecting the inherently interpretive nature of assessing teaching relevance: the criteria ask raters to judge how a misconception interacts with learners and instructional contexts they have experienced, and such judgments are shaped by individual teaching history. Notably, human reliability is close to the inter-model reliability reached in the LLM pilot ($\alpha = 0.65$ and $\alpha = 0.54$), suggesting that the definitions had reached a stable ceiling of agreement given the interpretive nature of the task — and supporting the use of LLM pilot ratings as a diagnostic tool for definitional stability. We therefore position the resulting ratings not as objective measurements but as structured heuristics that help instructors reason about priorities within a large collection.

Aggregation of Final Ratings. Final ratings for the repository were derived through a two-step aggregation procedure. Where two of three raters agreed and the third rating differed by at most one scale point, the majority rating was adopted, raising consistency to $\alpha = 0.90$ for Learner Discoverability and $\alpha = 1.00$ for Learning Barrier. In the four remaining cases of wider disagreement, the three instructors discussed the misconception in a consensus meeting and jointly agreed on a final assignment. This procedure yields final labels supported by either majority or consensus, and is reported here for transparency rather than as an additional reliability result.

Comparison with Student Tutor Ratings. To explore whether teaching experience specifically — as opposed to course familiarity — shapes teaching-relevance judgments, we conducted a parallel rating with three student tutors who had recently completed the course and supported it as teaching assistants. Tutor ratings showed moderate agreement for Learner Discoverability ($\alpha = 0.63$) but low agreement for Learning Barrier ($\alpha = 0.33$). While the small sample precludes strong claims, the asymmetry is consistent with the hypothesis that judging whether a misconception hinders long-term learning progress requires the kind of longitudinal perspective on student trajectories that develops through teaching practice, whereas estimating whether learners themselves notice a problem can be grounded in recent course experience. We therefore used only the instructor ratings for the final repository labels, and treat the tutor comparison as an exploratory observation that warrants follow-up with larger samples.

6 Rating Results and Distribution

Table 1 lists a representative selection of rated misconceptions with their final ratings for Learner Discoverability and Learning

Table 1: Teaching-relevance classifications (Low, Medium, High), including compound filter assignments (Extract; full set in the repository³)

Misconception	Discov. (L,M,H)	Barrier (L,H)	Frequ. (L,M,H)
Topic: Arrays			
Array Length As Number Top Priority Silent Blocker Common Obstacle Common Blind Spot Array lengths must be used as a constant number rather than using <code>.length</code> .	L	H	H
Array Length Only in Loop Conditions Common Obstacle The <code>.length</code> property can only be used in loop conditions.	M	H	H
Arrays Always Contain Data Obvious Pitfall Arrays automatically contain default values (even for objects).	H	H	L
One Loop Per Array Separate loops are required to traverse arrays of same length.	M	L	H
Array Indexing Needs Parentheses Array indexing needs extra parentheses around the index.	L	L	L
Arrays Empty At Beginning Arrays never contain any default data.	L	L	L
Topic: Functions			
Return At The End Top Priority Silent Blocker Common Obstacle Common Blind Spot A return can only be used at the end of a function.	L	H	H
No Function Calls Inside Functions Slow Burner Silent Blocker Functions cannot be called inside other functions.	L	H	M
Parameters Must Be Global Slow Burner Silent Blocker Parameters need to be defined globally.	L	H	M
One Return Per Function Rare Trap Silent Blocker Only one return can be used per function.	L	H	L
Return Outside Loop Rare Trap Silent Blocker A return can only be used outside a loop.	L	H	L
Return Multiple Values Obvious Pitfall Multiple values can be returned at the same time.	H	H	L
Data Types In Function Calls Obvious Pitfall Data types must be used on parameters when calling a function.	H	H	L
Signature Without Function Name The name of a function is not part of its signature.	M	H	M

Barrier, together with their observed frequency in student submissions; the complete set is available in the repository³. Frequency data is taken from our previous study [13], which analyzed 1,358 code artifacts and learning-related submissions from approximately 80 first-semester students in an introductory Processing course, complemented by 11 structured interviews. For each misconception, frequency was computed as its relative occurrence among all

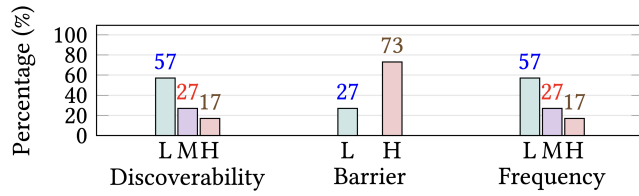


Figure 2: Distribution of Learner Discoverability, Learning Barrier, and observed Frequency (Low, Medium, High)

misconception occurrences within its conceptual category (arrays or functions), and mapped to three levels: Low ($< 5\%$), Medium ($5\text{--}15\%$), and High ($\geq 15\%$). The thresholds were chosen to separate marginal occurrences from misconceptions that appear in a substantial minority or majority of cases, and to yield a distribution balanced enough for filtering to be useful.

The resulting distribution (Figure 2) reveals three patterns with instructional implications. First, most misconceptions were rated as having low Learner Discoverability (57%) — meaning that the majority of documented misconceptions are ones that learners are unlikely to notice on their own. Second, a large majority were rated as high Learning Barriers (73%), indicating that most documented misconceptions substantially interfere with learning progress if left unaddressed. Together, these two patterns reinforce the core motivation for the repository: many misconceptions are both consequential and invisible to learners, making instructional attention essential. Third, most misconceptions occurred infrequently (57% Low Frequency), suggesting that any individual misconception typically affects a minority of students — but that the *cumulative* instructional burden across the collection is substantial.

The small proportion of high-Discoverability misconceptions likely reflects a property of the data source rather than the population of misconceptions at large: misconceptions that learners notice themselves tend to be resolved through in-session questions and therefore appear less often in submitted code artifacts. This is a limitation of deriving misconceptions primarily from static submissions and suggests that future extensions of the repository should incorporate complementary data sources such as help-seeking logs or classroom observations.

7 Navigating the Repository

The teaching-relevance ratings are made actionable through several navigation mechanisms grounded in the criteria above. The most targeted entry point is filtering by topic and subtopic. Topics correspond to broader conceptual areas such as arrays or functions; subtopics provide finer-grained groupings within a topic, such as *Return* or *Parameters & Arguments* within functions. A misconception can belong to multiple topics or subtopics, reflecting that some misconceptions span several conceptual areas. This supports a common instructional workflow: an instructor preparing a lesson can filter directly to the relevant subtopic and see which misconceptions are likely to arise.

Within a selected topic/subtopic, misconceptions can be sorted by Learner Discoverability, Learning Barrier, or observed frequency.

However, single-criteria sorting places the burden of interpretation on instructors, who must combine multiple signals to identify instructionally relevant misconceptions. The repository therefore provides **compound filters** that combine attributes into higher-level categories with direct instructional meaning. We focused on combinations that represent recurring teaching situations rather than enumerating the full attribute space. The three-criteria filters highlight misconceptions that are consequential and hard for learners to notice, differentiated by frequency: *Top Priorities* (Low Discoverability + High Barrier + High Frequency) are prime candidates for instructional intervention; *Slow Burners* (Low Discov. + High Barrier + Medium Frequency) significantly interfere with learning but accumulate gradually; *Rare Traps* (Low Discov. + High Barrier + Low Frequency) affect few learners but pose serious obstacles when they occur. The two-criteria filters target broader instructional situations: *Silent Blockers* (Low Discov. + High Barrier) — invisible but consequential; *Obvious Pitfalls* (High Discov. + High Barrier) — self-detected but disruptive; *Common Obstacles* (High Barrier + High Frequency) — frequent and high-impact; *Common Blind Spots* (Low Discov. + High Frequency) — frequent and undetected. Table 1 shows the resulting compound-filter assignments for a selection of misconceptions.

Beyond targeted filtering, the repository supports broader familiarization through alphabetical listings and a “daily misconception” random view that encourages incremental engagement with the collection over time.

8 Instructional Use Scenarios

The navigation mechanisms are designed to support two primary use cases. For **lesson preparation**, an instructor selects the topic/subtopic currently covered, applies a compound filter to identify the most instructionally relevant misconceptions, and uses the associated teaching opportunities — such as peer instruction questions or short exercises — to prepare targeted interventions. *Silent Blockers* is particularly useful here, surfacing misconceptions students are unlikely to detect on their own but that strongly hinder learning progress. Return At The End, for instance — the example from the introduction — surfaces immediately when filtering the subtopic Return: frequent, hard to detect, and a strong barrier to conditional logic. Sorting by frequency further helps decide which misconceptions to emphasize in a session. The *Common Obstacles* filter can also inform exam design by highlighting misconceptions that are both frequent and consequential. For **tutor training**, the repository serves as a structured orientation tool. Tutors often have strong content knowledge but limited experience recognizing misconceptions. Filtering by topic and applying *Common Blind Spots* lets tutors focus on misconceptions they are most likely to encounter in lab sessions, while documented origins and typical errors support a shared understanding of why students struggle. The teaching opportunities associated with each entry can serve as starting points for practice scenarios.

9 Discussion

Building an instructor-oriented repository involves design decisions that go beyond collecting misconceptions. We reflect on two

decisions here: how to define teaching-relevance criteria that multiple raters can apply consistently, and how to integrate them into navigation that reduces rather than adds interpretive burden.

Regarding RQ1 — whether teaching-relevance criteria can be defined and rated reliably — LLM-based pilot ratings proved useful for surfacing definitional ambiguities and for identifying dimensions that were too context-dependent or insufficiently separable, without compromising human rater independence. The two retained criteria reached moderate human inter-rater reliability ($\alpha = 0.57$ and $\alpha = 0.60$) — values close to the inter-model reliability reached at the end of the LLM pilot ($\alpha = 0.65$ and $\alpha = 0.54$). We interpret this convergence as indicating that the definitions had reached a stable ceiling of agreement given the interpretive nature of the task, and as supporting the use of LLMs as diagnostic tools for definitional clarity. We therefore treat the resulting ratings not as objective measurements but as structured heuristics: instructional relevance is inherently context-dependent, and the ratings help instructors reason about priorities without claiming to reduce relevance to a single fixed value. The exploratory comparison with student tutor ratings is consistent with the view that judging long-term instructional impact draws on a form of pedagogical content knowledge that develops through teaching rather than course participation alone — though the small sample size precludes strong claims.

Regarding RQ2 — how teaching-relevance criteria and compound filters can be integrated to support instructional decision-making — single ratings place the burden of interpretation on the instructors, who must combine multiple signals to identify what matters. Compound filters address this by packaging recurring teaching situations — *Silent Blockers*, *Common Obstacles*, *Top Priorities* — into directly actionable categories. Combined with topic and subtopic filtering, they support a natural instructional workflow: narrow to the relevant conceptual area, then surface the most instructionally critical misconceptions. Although the current repository is grounded in Processing, most documented misconceptions transfer directly to Java and, more broadly, to languages with similar semantics, suggesting that the overall approach generalizes beyond a single language or course.

The central limitation of this work is the absence of a formal usability study: claims about effectiveness rest on design intent and formative teaching observations, not on evidence of how instructors and tutors actually use the repository. The human rating sample is also small (three instructors), and the frequency data derives from a single introductory course — both constraints that should be addressed as the repository expands.

10 Conclusion and Future Work

Misconception collections only realize their potential when educators can navigate them efficiently and translate entries into teaching decisions. We present an instructor-oriented repository for Java/Processing that combines two teaching-relevance criteria (*Learner Discoverability* and *Learning Barrier*) with observed frequency data into sorting, topic filtering, and seven compound filters. The criteria were developed through an iterative LLM-based pilot process that validated definitions before involving human raters — a methodological contribution that may transfer to other

annotation tasks where rater availability is scarce. To our knowledge, teaching-relevance criteria, empirical frequency data, and compound filters are absent from existing collections, making instructional prioritization a distinctive contribution of this work.

Future work will address three directions: systematic usability evaluation with instructors and tutors; extension of the teaching-relevance criteria across educational contexts and cohort levels, potentially revisiting dimensions excluded here; and collaboration with the wider community — including the ProgMiscon [3] project — to exchange misconception entries and explore LLM-based detection of misconceptions in student code.

Acknowledgments

This work is funded by the KI-AUX project, supported by the Stiftung Innovation in der Hochschullehre (Foundation for Innovation in Higher Education).

References

- [1] Ricardo Caceffo, Pablo Frank-Bolton, Renan Souza, and Rodolfo Azevedo. 2019. Identifying and Validating Java Misconceptions Toward a CS1 Concept Inventory. In *Proceedings of the 2019 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '19)*. Association for Computing Machinery, New York, NY, USA, 23–29. doi:10.1145/3304221.3319771
- [2] Luca Chiodini, Joey Bevilacqua, and Matthias Hauswirth. 2025. Surveying Upper-Secondary Teachers on Programming Misconceptions. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1 (ICER '25)*. Association for Computing Machinery, New York, NY, USA, 166–180. doi:10.1145/3702652.3744227
- [3] Luca Chiodini, Igor Moreno Santos, Andrea Gallidabino, Anya Tafilovich, André L. Santos, and Matthias Hauswirth. 2021. A Curated Inventory of Programming Language Misconceptions. In *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '21)*. Association for Computing Machinery, New York, NY, USA, 380–386. doi:10.1145/3430665.3456343
- [4] Osvaldo Clua and Maria Feldgen. 2008. Work in progress - object oriented metrics and programmers' misconceptions. In *2008 38th Annual Frontiers in Education Conference*. IEEE, Piscataway, NJ, USA, F3H–19–F3H–20. doi:10.1109/FIE.2008.4720377 ISSN: 2377-634X.
- [5] Louis Deslauriers, Ellen Schelew, and Carl Wieman. 2011. Improved Learning in a Large-Enrollment Physics Class. *Science* 332, 6031 (May 2011), 862–864. doi:10.1126/science.1201783
- [6] John Dewey. 2015. *Experience and education* (first free press edition 2015 ed.). Free Press, New York London Toronto Sydney New Delhi.
- [7] Dimitri Eckert, Dion Timmermann, and Christian Kautz. 2022. Student Misconceptions about Loops in Introductory Programming Courses and the Influence of Representations. In *2022 IEEE Frontiers in Education Conference (FIE)*. IEEE, Uppsala, Sweden, 1–5. doi:10.1109/FIE56618.2022.9962545
- [8] K. Anders Ericsson, Ralf T. Krampe, and Clemens Tesch-Römer. 1993. The role of deliberate practice in the acquisition of expert performance. *Psychological Review* 100, 3 (1993), 363–406. doi:10.1037/0033-295X.100.3.363 Place: US Publisher: American Psychological Association.
- [9] Sally Fincher, Johan Jeuring, Craig S. Miller, Peter Donaldson, Benedict du Boulay, Matthias Hauswirth, Arto Hellas, Feliene Hermans, Colleen Lewis, Andreas Mühling, Janice L. Pearce, and Andrew Petersen. 2020. Notional Machines in Computing Education: The Education of Attention. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '20)*. Association for Computing Machinery, New York, NY, USA, 21–50. doi:10.1145/3437800.3439202
- [10] John Hattie. [n.d.]. Visible Learning—Jigsaw method Details. https://www.visiblelearningmetax.com/influences/view/jigsaw_method Webpage (no publication date provided).
- [11] Lea Jell, Corinna List, and Michael Kipp. 2023. Towards Automated Interactive Tutoring - Focussing on Misconceptions and Adaptive Level-Specific Feedback. In *Proceedings of the 5th European Conference on Software Engineering Education (ECSEE '23)*. Association for Computing Machinery, New York, NY, USA, 226–235. doi:10.1145/3593663.3593692
- [12] Lisa C. Kaczmarczyk, Elizabeth R. Petrick, J. Philip East, and Geoffrey L. Herman. 2010. Identifying student misconceptions of programming. In *Proceedings of the 41st ACM technical symposium on Computer science education (SIGCSE '10)*. Association for Computing Machinery, New York, NY, USA, 107–111. doi:10.1145/1734263.1734299

- [13] Corinna List, Moritz Müller, and Michael Kipp. 2025. Yet Another Collection of Programming Misconceptions – To Help Educators Find What Matters. In *Proceedings of the 25th Koli Calling International Conference on Computing Education Research (ACM Other conferences)*. ACM, New York, NY, USA, 1–3. doi:10.1145/3769994.3770058
- [14] E. Mazur. 1997. *Peer Instruction: A User's Manual*. Prentice Hall, Upper Saddle River, NJ, USA.
- [15] Yizhou Qian and James Lehman. 2018. Students' Misconceptions and Other Difficulties in Introductory Programming: A Literature Review. *ACM Transactions on Computing Education* 18, 1 (March 2018), 1–24. doi:10.1145/3077618
- [16] Juha Sorva. 2012. *Visual Program Simulation in Introductory Programming Education*. Ph.D. Dissertation. Aalto University, Espoo, Finland.
- [17] Benedikt Wisniewski, Klaus Zierer, and John Hattie. 2020. The Power of Feedback Revisited: A Meta-Analysis of Educational Feedback Research. *Frontiers in Psychology* 10 (2020). doi:10.3389/fpsyg.2019.03087